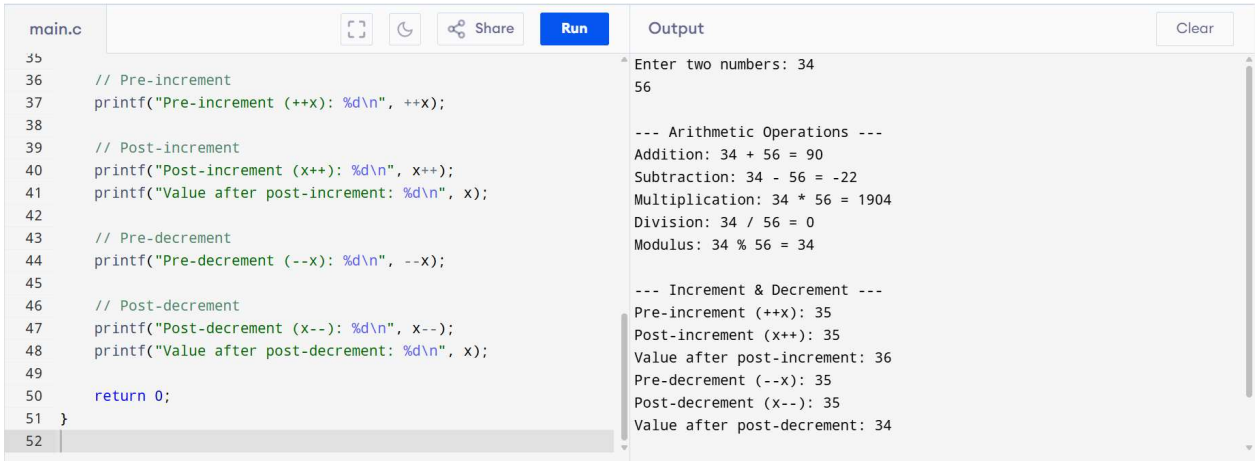
1.A code that covers all arithmetic operators including increment and decrement.

```
#include <stdio.h>

int main() {
    int a, b;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    printf("\n--- Arithmetic Operations ---\n");

    // Addition
    printf("Addition: %d + %d = %d\n", a, b, a + b);

    // Subtraction
    printf("Subtraction: %d - %d = %d\n", a, b, a - b);

    // Multiplication
    printf("Multiplication: %d * %d = %d\n", a, b, a * b);

    // Division
    if (b != 0)
        printf("Division: %d / %d = %d\n", a, b, a / b);
    else
        printf("Division: Cannot divide by zero\n");

    // Modulus
    if (b != 0)
        printf("Modulus: %d %% %d = %d\n", a, b, a % b);
    else
        printf("Modulus: Cannot divide by zero\n");

    printf("\n--- Increment & Decrement ---\n");

    int x = a;

    // Pre-increment
    printf("Pre-increment (++x): %d\n", ++x);

    // Post-increment
    printf("Post-increment (x++): %d\n", x++);
    printf("Value after post-increment: %d\n", x);
```

```
    // Pre-decrement
    printf("Pre-decrement (--x): %d\n", --x);

    // Post-decrement
    printf("Post-decrement (x--): %d\n", x--);
    printf("Value after post-decrement: %d\n", x);

    return 0;
}
```



Here are your **Logical Operators notes (exam-ready format)**:

Logical Operators – Notes

Definition

Logical operators are used to **combine multiple conditions** and return a boolean result:

- **True (1)**
- **False (0)**

They are mainly used in **decision-making statements** like `if`, `while`, etc.

Types of Logical Operators

1. AND Operator (&&)

- Returns **true only if both conditions are true**
- If any one condition is false → result is false

Example:

```
int x = 6, y = 3;
if (x < 10 && y > 1)
   printf("True");
```

Output: True

---

2. OR Operator (| |)

- Returns **true if at least one condition is true**
- Returns false only if both conditions are false

Example:

```
int x = 6, y = 3;
if (x == 5 || y == 5)
   printf("True");
else
   printf("False");
```

Output: False

---

3. NOT Operator (!)

- Used to **reverse the result**
- True becomes false, false becomes true

Example:

```
int x = 6, y = 3;
if (!(x == y))
   printf("True");
```

Output: True

Truth Table

A	B	A && B	A || B
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

NOT Operator:

A	!A
0	1
1	0

2.Code for logical operators:-

```
#include <stdio.h>

int main() {
    int a, b;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    printf("\n--- Logical Operators ---\n");

    // AND operator
    if (a > 0 && b > 0)
        printf("AND (a > 0 && b > 0): True\n");
    else
        printf("AND (a > 0 && b > 0): False\n");

    // OR operator
    if (a > 0 || b > 0)
```

```
        printf("OR (a > 0 || b > 0): True\n");
    else
        printf("OR (a > 0 || b > 0): False\n");

    // NOT operator
    if (!(a > b))
        printf("NOT (!(a > b)): True\n");
    else
        printf("NOT (!(a > b)): False\n");

    return 0;
}
```

main.c Share Run Output Clear

```
        printf("AND (a > 0 && b > 0): True\n");
    else
        printf("AND (a > 0 && b > 0): False\n");

    // OR operator
    if (a > 0 || b > 0)
        printf("OR (a > 0 || b > 0): True\n");
    else
        printf("OR (a > 0 || b > 0): False\n");

    // NOT operator
    if (!(a > b))
        printf("NOT (!(a > b)): True\n");
    else
        printf("NOT (!(a > b)): False\n");

    return 0;
}
```

```
Enter two numbers: 45
67

--- Logical Operators ---
AND (a > 0 && b > 0): True
OR (a > 0 || b > 0): True
NOT (!(a > b)): True


=== Code Execution Successful ===
```

Comparison operators:-

Operator	Meaning
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to
==	Equal to
! =	Not equal to

3.Code for all comparison operators.

```
#include <stdio.h>

int main() {
    int a, b;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    printf("\n--- Comparison Operators ---\n");

    // Equal to
    printf("%d == %d : %d\n", a, b, a == b);

    // Not equal to
    printf("%d != %d : %d\n", a, b, a != b);

    // Greater than
    printf("%d > %d : %d\n", a, b, a > b);

    // Less than
    printf("%d < %d : %d\n", a, b, a < b);

    // Greater than or equal to
    printf("%d >= %d : %d\n", a, b, a >= b);

    // Less than or equal to
    printf("%d <= %d : %d\n", a, b, a <= b);

    return 0;
}
```

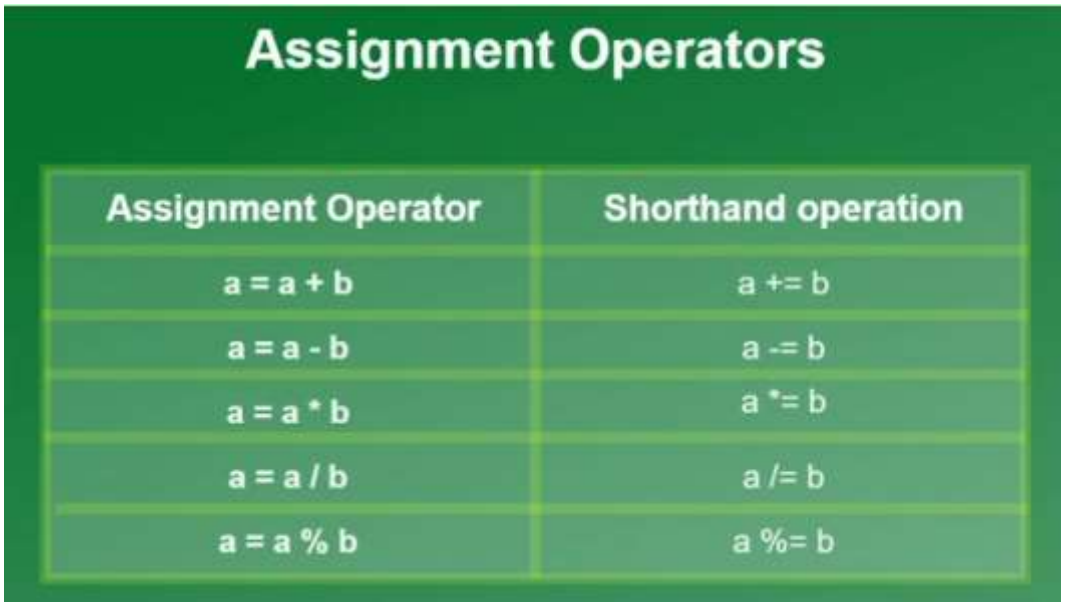
main.c

```c
    // Not equal to
    printf("%d != %d : %d\n", a, b, a != b);

    // Greater than
    printf("%d > %d : %d\n", a, b, a > b);

    // Less than
    printf("%d < %d : %d\n", a, b, a < b);

    // Greater than or equal to
    printf("%d >= %d : %d\n", a, b, a >= b);

    // Less than or equal to
    printf("%d <= %d : %d\n", a, b, a <= b);

    return 0;
}
```

Output

```
Enter two numbers: 34
67

--- Comparison Operators ---
34 == 67 : 0
34 != 67 : 1
34 > 67 : 0
34 < 67 : 1
34 >= 67 : 0
34 <= 67 : 1


=== Code Execution Successful ===
```

Assignment operators:-



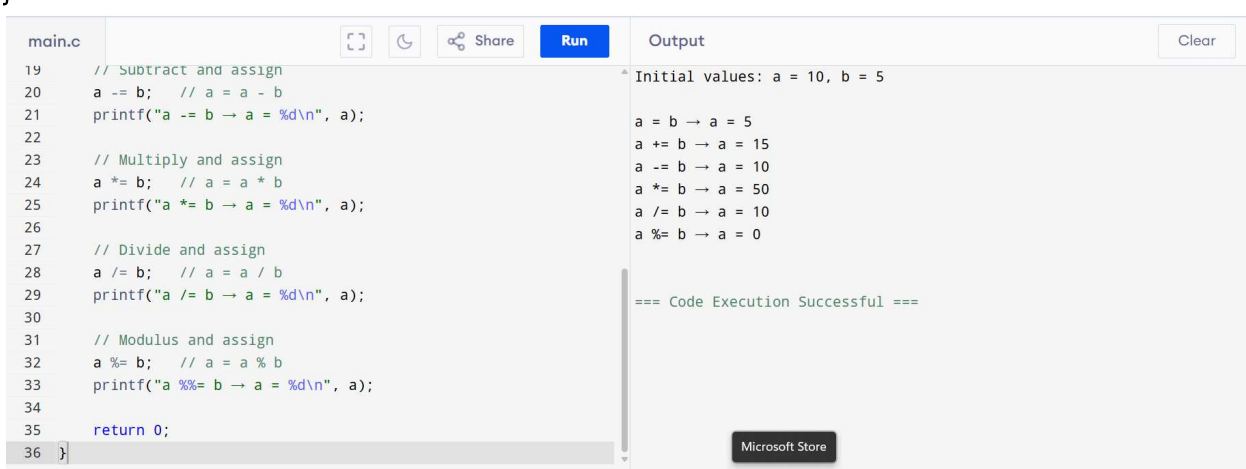
Assignment Operators

Assignment Operator	Shorthand operation
a = a + b	a += b
a = a - b	a -= b
a = a * b	a *= b
a = a / b	a /= b
a = a % b	a %= b

4.Code for assignment operators.

```
#include <stdio.h>

int main() {
    int a = 10, b = 5;

    printf("Initial values: a = %d, b = %d\n", a, b);

    // Simple assignment
    a = b;
    printf("\na = b → a = %d\n", a);
```

```
    // Reset value
    a = 10;

    // Add and assign
    a += b;   // a = a + b
    printf("a += b → a = %d\n", a);

    // Subtract and assign
    a -= b;   // a = a - b
    printf("a -= b → a = %d\n", a);

    // Multiply and assign
    a *= b;   // a = a * b
    printf("a *= b → a = %d\n", a);

    // Divide and assign
    a /= b;   // a = a / b
    printf("a /= b → a = %d\n", a);

    // Modulus and assign
    a %= b;   // a = a % b
    printf("a %%= b → a = %d\n", a);

    return 0;
}
```



Precedence and Associativity of operators:-

OPERATOR	TYPE	ASSOCIAVITY
( ) [ ] . ->		left-to-right
++ -- + - ! ~ (*type*) * & sizeof	Unary Operator	right-to-left
* / %	Arithmetic Operator	left-to-right
+ -	Arithmetic Operator	left-to-right
<< >>	Shift Operator	left-to-right
< <= > >=	Relational Operator	left-to-right
== !=	Relational Operator	left-to-right
&	Bitwise AND Operator	left-to-right
^	Bitwise EX-OR Operator	left-to-right
|	Bitwise OR Operator	left-to-right
&&	Logical AND Operator	left-to-right
||	Logical OR Operator	left-to-right
? :	Ternary Conditional Operator	right-to-left
= += -= *= /= %= &= ^= |= <<= >>=	Assignment Operator	right-to-left
,	Comma	left-to-right

5.Code for Precedence and Associativity of operators

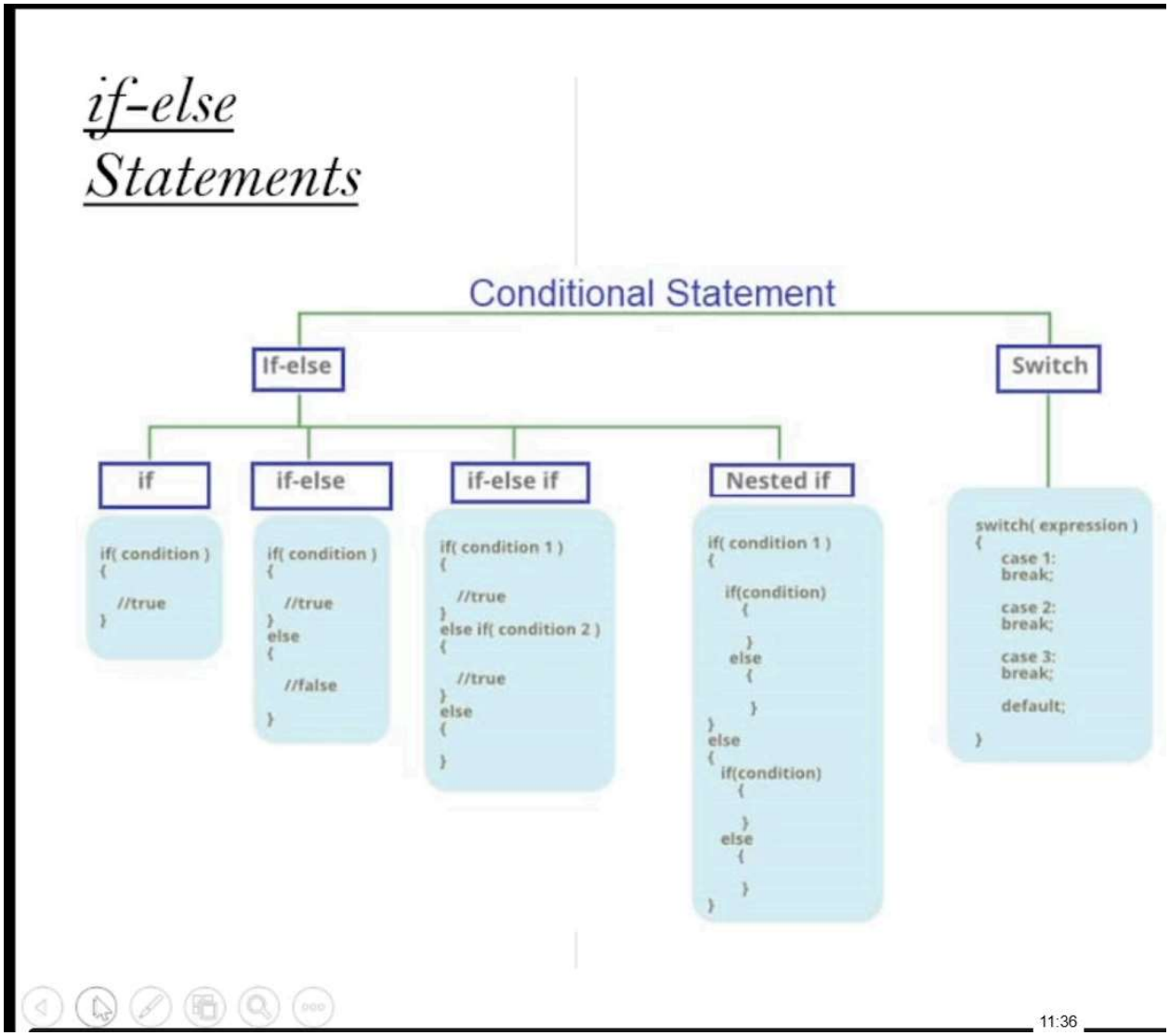
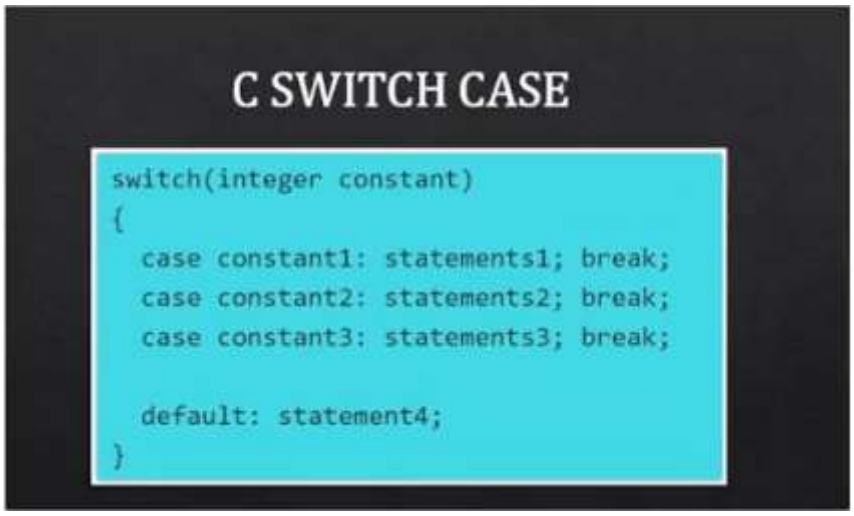
```
#include <stdio.h>

int main() {
   int a = 10, b = 5, c = 2, result;

   printf("Values: a = %d, b = %d, c = %d\n\n", a, b, c);
```

```
    // Precedence example
    result = a + b * c;
    printf("a + b * c = %d\n", result);
    // Multiplication (*) has higher precedence than addition (+)

    // Using parentheses to change precedence
    result = (a + b) * c;
    printf("(a + b) * c = %d\n", result);

    // Associativity example (left to right)
    result = a - b - c;
    printf("a - b - c = %d\n", result);
    // Evaluated as (a - b) - c

    // Assignment associativity (right to left)
    int x, y;
    x = y = 20;
    printf("x = y = 20 → x = %d, y = %d\n", x, y);

    // Mixed operators
    result = a + b * c - a / c;
    printf("a + b * c - a / c = %d\n", result);

    return 0;
}
```

```
14      result = (a + b) * c;
15      printf("(a + b) * c = %d\n", result);
16
17      // Associativity example (left to right)
18      result = a - b - c;
19      printf("a - b - c = %d\n", result);
20      // Evaluated as (a - b) - c
21
22      // Assignment associativity (right to left)
23      int x, y;
24      x = y = 20;
25      printf("x = y = 20 → x = %d, y = %d\n", x, y);
26
27      // Mixed operators
28      result = a + b * c - a / c;
29      printf("a + b * c - a / c = %d\n", result);
30
31      return 0;
```

```
Values: a = 10, b = 5, c = 2

a + b * c = 20
(a + b) * c = 30
a - b - c = 3
x = y = 20 → x = 20, y = 20
a + b * c - a / c = 15


=== Code Execution Successful ===
```

If-else statements:-



Switch-case statements:-